



수학 시뮬레이터 제작 보고서

수행평가

《시뮬레이션 파일 저장하는 방법》

- ① 최종 시뮬레이터의 html 코드를 전체 복사한다.
- ② 윈도우 메모장을 켜 후 위의 코드를 붙여넣기 한다.
- ③ 메모장에서 파일 - 다른 이름으로 저장 - 각각 본인이름1.html, 본인이름2.html으로 저장
(예 : 이현진1.html, 이현진2.html으로 꼭 확장자 붙여서 저장)

[첫 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 첫 번째 문제 (캡처해서 붙여넣기 해도 됨)

IASA 학동 (2026-1-9) 학번: 이름:

문제 4 2023년 서울대학교 기술 재확인 CHECK

10원짜리, 100원짜리, 500원짜리 동전이 각각 하나씩 놓여있다. 차례로 동전을 한 개씩 뒤집는 작업을 통해 동전을 다음의 상태로 바꾸려고 한다.

(*) 3개의 동전이 모두 앞면이거나 모두 뒷면
동전을 뒤집을 순서를 차례대로 나열한 수열을 '뒤집기 수열'이라고 하자. 즉, '뒤집기 수열' $\{a_n\}$ 은 n 번째에 a_n 원짜리 동전을 뒤집는 것을 말하며 수열 $\{a_n\}$ 의 모든 항은 10, 100, 500 중 하나이다. 예를 들어, '뒤집기 수열' 100, 500, ... 에 따라 동전을 뒤집으면 다음과 같다.

1번째 뒤집기 2번째 뒤집기

10원 100원 500원 → 10원 100원 500원 → 10원 100원 500원 → ...

○ : 앞면 ● : 뒷면

도전자가 '뒤집기 수열'을 하나 제시하면, 심판이 3개의 동전을 (*) 상태가 아니도록 무작위로 놓은 후, 도전자가 제시한 '뒤집기 수열'에 따라 동전을 뒤집는다. 3개의 동전이 (*) 상태가 되면 뒤집기를 멈춘다.

- (1) 모든 '뒤집기 수열'에 대해 1번 만에 3개의 동전이 (*) 상태로 바뀔 확률은 같다. 그 확률을 구하시오.
- (2) 2번 이내에 3개의 동전이 (*) 상태로 바뀔 확률을 최대로 만드는 '뒤집기 수열' 하나의 처음 두 개 항을 제시하고 그 최대의 확률을 구하시오.
- (3) n 번 이내에 3개의 동전이 (*) 상태로 바뀔 확률이 1인 '뒤집기 수열'이 존재하도록 하는 n 의 최솟값을 구하시오.
- (4) 위에서 (*) 상태를 아래의 (**) 상태로 대체한다.
(**) 3개의 동전이 모두 앞면
도전자가 '뒤집기 수열'을 하나 제시하면, 심판이 3개의 동전을 (**) 상태가 아니도록 무작위로 놓은 후, 도전자가 제시한 '뒤집기 수열'에 따라 동전을 뒤집는다. 3개의 동전이 (**) 상태가 되면 뒤집기를 멈춘다. n 번 이내에 3개의 동전이 (**) 상태로 바뀔 확률이 1인 '뒤집기 수열'이 존재하도록 하는 n 의 최솟값을 구하시오.

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

문 4.

(1) 반드시 3개 동전 중 1개는 반대면
ex) TTHH
이 경우, (*)을 만족할 확률은 $\frac{1}{3}$

(2) WLOG, 10원을 뒤집는다 가정.
TTHH HTHH HHTT
HTTT THTT TTTH
/: 1회 \: 2회
(10원 → 100원 순)
 $\therefore \frac{2}{3}$

(3) $a \rightarrow b \rightarrow a$ 꼴로 뒤집을 시 반드시 해결 가능
($\because a \rightarrow b$ 했을 때 완료 x면 c만 다른 색)
 $\Rightarrow 3$

(4) 전체 상태 수: $2^3 = 8$ 가지
첫 번째에는 반드시 (**)이 아님
 $\therefore 2^3 - 1 = 7$ (번)

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

본 문제는 얼핏 간단해 보이는 동전 뒤집기 규칙을 갖고 있으나, 시행 횟수가 증가함에 따라 발생할 수 있는 상태 전이의 가짓수가 기하급수적으로 늘어나는 복잡성을 지니고 있다.

이 문제에서 시뮬레이션이 필수적이라고 느낀 이유는 우선 인간이 가진 직관의 한계를 극복하고 엄밀한 반례 검증을 수행하기 위해서이다. 문항 (3)과 (4)에서 요구하는 '확률이 1이 되는 최소 단계 n'을 구하기 위해서는 단순히 성공하는 몇 가지 케이스를 찾는 것을 넘어, 6가지 모든 초기 상태에서 단 하나의 예외나 무한 루프에 빠지는 반례 수열 없이 무조건 목표 상태에 도달하는가를 완벽하게 증명해야 한다. 인간의 제한된 수리적 직관만으로는 뒤집기 수열이 길어질 때 나타나는 사각지대를 모두 파악하기 어렵기 때문에, 모든 독립적인 경우의 수를 완벽하게 추적해 줄 컴퓨터 알고리즘이 필요했다.

또한 문항 (4)처럼 목표 상태가 모든 동전이 앞면으로 제한되는 경우, 탐색해야 하는 뒤집기 수열의 깊이가 늘어나면서 전수조사를 직접 수행하는 데 물리적인 한계에 부딪히게 된다. 이를 인간이 직접 손으로 수행도를 그려 조사하려면 수백 가지 조합을 일일이 계산해야 하므로 연산 실수의 확률이 매우 높아진다. AI를 통해 제작한 시뮬레이터는 이러한 물리적인 계산 피로도를 완전히 배제하고 정확한 데이터 기반의 힌트를 실시간으로 제공해 주는 역할을 한다. 마지막으로 정적인 텍스트 풀이와 달리 시뮬레이터는 임의로 조합한 뒤집기 수열에 따라 6가지 서로 다른 초기 상태들이 동시에 어떻게 변해가는지 실시간 상태 대시보드로 보여준다. 이 동적인 피드백 과정을 거치면서 동전을 중복하여 뒤집지 않고 순환시킬 때 상태 전이가 가장 효율적으로 일어난다는 수학적 아이디어를 시각적으로 확신할 수 있었고, 문제의 기하학적 구조를 이해하는 데 큰 도움이 되었다.

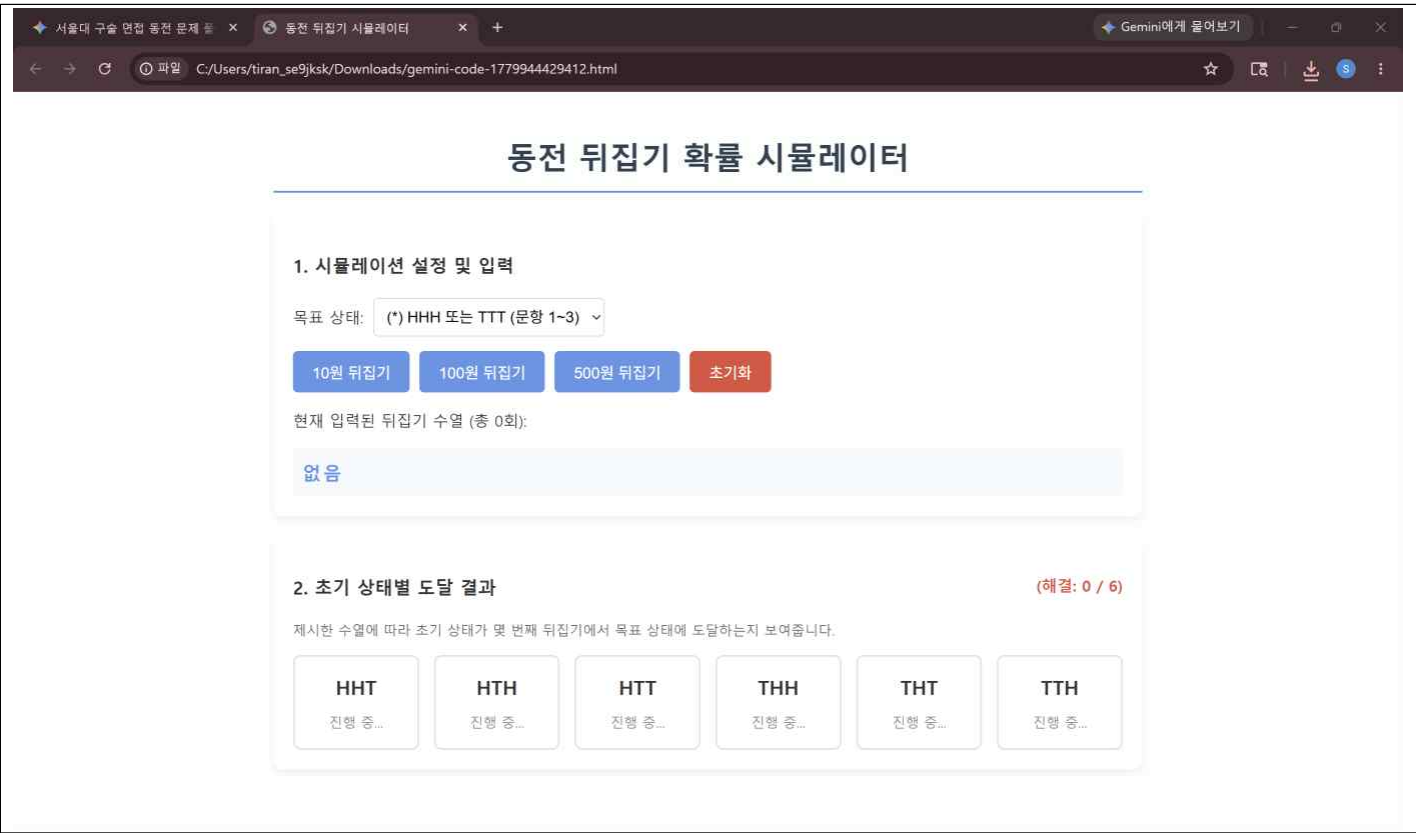
4. 사용한 AI 플랫폼을 모두 적으세요.

Gemini

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

이 문제를 해결하는 데에 필요한 정보를 얻게 해주는 시뮬레이션을 html로 작성하고 싶은데, 계획을 로드맵으로 세워줘. 그 다음 작성한 로드맵의 내용을 바탕으로 시뮬레이터를 작성해줘. 단, 결과물은 반드시 html 파일 1개로 나와야만 해.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

먼저, 수정이 필요하다고 여겨지는 부분은 경우의 수 탐색의 자동화 기능이 부족한 것이다. 최초 시뮬레이터는 사용자가 화면의 버튼을 직접 클릭하여 뒤집기 수열을 수동으로 입력하는 방식으로 설계되어 있다. 이로 인해 시행 횟수 n 이 커짐에 따라 기하급수적으로 증가하는 모든 뒤집기 수열의 조합을 인간이 일일이 대조하고 검증하는 데 한계가 있다. 특정 단계 n 을 입력하면 컴퓨터가 가능한 모든 수열 조합을 연산하여 최적의 해를 찾아내는 백그라운드 자동 연산 알고리즘과 전수조사 엔진의 도입이 시급하다.

둘째로는 정보 시각화의 공간 효율성과 사용성 측면에서 개선이 필요하다. 초기 상태 6가지에 대한 도달 현황이 화면을 크게 차지하는 그리드 카드 형태로 나열되어 있어, 탐색 과정과 결과를 한눈에 통합적으로 모니터링하기 어렵다. 연산 진행 상황, 현재 검사 중인 수열, 발견된 최적의 수열 목록 등을 터미널 형태의 독립된 텍스트 로그 영역으로 분리하여 출력함으로써 데이터의 가독성과 모니터링의 효율성을 높여야 한다.

더불어, 문제 조건 변화에 따른 유연한 대응 능력이 결여되어 있다. 최초 시뮬레이터는 동전 상태가 모두 같아지는

상태만을 판별하도록 고정되어 있어, 목표 상태가 모든 동전이 앞면(HHH)으로 제한되는 심화 문항 조건을 실험하기 어렵다. 사용자가 드롭다운 메뉴 등을 통해 목표 조건을 동적으로 선택하고, 변경된 조건에 따라 성공 확률 분모와 연산 구조가 유연하게 가변되도록 확장성을 확보해야 한다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

(2번째)

현재 시뮬레이션만으로는 왜 답의 확률이나 n 의 최솟값이 그렇게 도출되는 지 알 수 없어. 해당 정보들을 알 수 있도록 시행이 자동으로 빠르게 진행되도록 하는 기능을 추가해줘.

(3번째)

이제 디자인을 수정해보자. 아래 프롬프트를 참고하되, 이는 apple 제품 소개의 프롬프트이니 완벽하게 반영하려 하지 마.

프로그램은 반드시 이전과 동일한 기능을 갖추고 동작하여야 해.

(파일 첨부하였음)

(4번째)

파일에 존재하는 모든 텍스트의 괄호를 통한 영어 병기를 삭제해.

(5번째)

이건 내가 간단하게 작성한 '앞면에는 H, 뒷면에는 T가 적힌 동전이 공중에서 회전하는 애니메이션 html 코드'인데, 시뮬레이션 웹페이지의 한 부분에 첨가해줄 수 있어? 넣는 위치는 우측의 하얀 컨테이너 아래로 해줘. 초기 상태별 도달 현황의 6가지는 크기를 조절하여 한 줄에 들어올 수 있도록 해줘.

물론 기존의 기능은 그대로 동작해야해.

(파일 첨부하였음)

(6번째)

애니메이션의 크기가 너무 크다. 조금만 동전 애니메이션의 크기를 줄여줘. 하얀 컨테이너 위로 옮기자.

(7번째)

전체 화면을 했을 때, 화면 안에 전부 안들어오는데, 이것을 수정해줄래? 또한, 자동 탐색의 횟수가 길어지면 모든 경우의 수 실행이 오래걸리니까 중간에 초기화할 수 있는 버튼이 필요할 것 같아.(모든 경우의 수 실행 버튼을 실행 중에는 초기화 버튼으로 바뀌게 해줘)

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

시뮬레이션 엔진

수학으로 증명하는 확률 모델링

서울대학교 기술문제 기반 동전 뒤집기 알고리즘의 최적 해를 탐색합니다. 직접 데이터를 주입하거나, 인공지능이 모든 경우의 수를 시뮬레이션하도록 지시하십시오.

목표 상태

HHH 또는 TTT 도달

수동 제어

10

100

500

초기화

자동 탐색

3

모든 경우의 수 실행



현재 입력 수열

대기 중

초기 상태별 도달 현황

0 / 6

HHT

대기

HTH

대기

HTT

대기

THH

대기

THT

대기

TTH

대기

시스템 대기 중. 수동으로 조작하거나 시뮬레이션을 시작하십시오.

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

본 시뮬레이터의 화면은 크게 좌측의 제어 및 입력 패널과 우측의 실시간 시각화 및 터미널 패널로 구성되어 있으며, 처음 사용하는 사람도 아래의 4단계 순서에 따라 쉽게 조작하고 수학적 힌트를 얻을 수 있다.

1. 시뮬레이션 목표 및 조건 설정

실험을 시작하기 전, 좌측 첫 번째 목표 상태 섹션의 드롭다운 메뉴를 클릭하여 탐색하고자 하는 문항의 조건을 선택한다.

- 'HHH 또는 TTT 도달'을 선택하면 세 동전의 면이 모두 같아지는 상태를 목표로 잡는다.
- 'HHH 상태만 도달'을 선택하면 오직 세 동전이 모두 앞면이 되는 상태만을 성공 조건으로 제한한다.

조건을 변경하면 기존에 화면에 입력되어 있던 모든 수열과 결과 데이터는 안전하게 자동 초기화된다.

2. 수동 제어로 나만의 뒤집기 수열 실험하기

사용자가 직접 논리적으로 구상한 수열을 주입하여 동전의 상태 변화를 단계별로 추적할 수 있다.

- '수동 제어' 섹션에 배치된 '10', '100', '500' 버튼을 원하는 순서대로 클릭하면, 해당 금액의 동전이 뒤집기 수열에 차례대로 누적된다.
- 버튼을 클릭할 때마다 우측 상단의 '현재 입력 수열' 대시보드에 내가 입력한 동전의 종류가 화살표(→) 기호와 함께 실시간으로 나열된다.
- 만약 수열을 잘못 입력했거나 새로운 규칙으로 처음부터 다시 설계하고 싶다면, 빨간색 글씨의 '초기화' 버튼을 눌러 수열을 비울 수 있다.

3. 실시간 도달 현황 및 3D 애니메이션 모니터링

수열이 입력됨에 따라 우측 화면의 시각화 장치들이 실시간 수학적 연산 결과를 직관적으로 보여준다.

- 패널 상단에서는 앞면과 뒷면이 교차하는 3D 동전 오브젝트가 공중에서 역동적으로 회전하며 시각적 몰입감을 더한다.
- 초기 상태별 도달 현황 영역에서는 문제에서 제시한 6가지 초기 상태(HHT, HTH, HTT, THH, THT, TTH)가 현재 입력된 수열에 의해 목표 상태에 도달했는지 보여준다. 수열의 중간 단계에서라도 목표에 도달하는 데 성공한 카드는 연두색 배경으로 반전되며 n회차라는 완료 표기가 나타나고, 도달하지 못한 상태는 대기 상태로 남아있게 된다. 또한 우측 상단의 스코어보드를 통해 전체 초기 상태 중 몇 개가 클리어되었는지 한눈에 파악할 수 있다.

4. 자동 탐색 엔진으로 최적의 수열과 최솟값 n 추적

인간이 전수조사하기 어려운 긴 수열에 대해 컴퓨터 알고리즘을 이용하여 확률이 1이 되는 최적의 해를 초고속으로 찾아낼 수 있다.

- '자동 탐색' 섹션의 숫자 입력창에 검증하고 싶은 수열의 길이 n을 입력한다.
- '모든 경우의 수 실행' 버튼을 누르면 시스템이 백그라운드에서 가능한 모든 수열 조합을 체크 단위로 분할하여 연산하기 시작한다.
- 연산이 진행되는 동안 우측 하단의 검은색 터미널 로그 창을 통해 현재까지의 전체 연산 진행률, 실시간으로 검사 중인 수열 상태, 현재까지 발견된 최대 도달 확률 및 발견된 최적 수열 목록이 실시간으로 스크롤되며 출력된다.
- 만약 연산 과정이 너무 길어지거나 도중에 탐색을 멈추고 싶다면, 실행 중 빨간색으로 변경된 초기화 및 중단 버튼을 클릭하여 즉시 시스템을 안전하게 대기 상태로 되돌릴 수 있다.
- 자동 탐색이 성공적으로 완료되면, 엔진이 찾아낸 최적의 수열 중 가장 첫 번째 조합이 화면에 자동으로 주입되어 상단의 도달 현황판에 완벽하게 시각화된다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

[도움이 된 부분]

구상했던 수학적 아이디어를 복잡한 수작업 없이 초고속으로 전수조사하여 완벽하게 검증할 수 있었다. 문항 (3)에서 모든 초기 상태를 3번의 시행 안에 해결할 수 있다는 가설을 자동 탐색 엔진을 통해 검증하였고, 연산 결과가 데이터로그로 즉각 출력되어 풀이의 확신을 얻었다. 특히 손으로 직접 수형도를 그리기에 엄두가 나지 않던 문항 (4)의 심화 조건에 대해서도 자동 탐색 알고리즘을 통해 최솟값이 7이라는 명확한 정답과 최적 수열이라는 결정적인 힌트를 실시간으로 얻어 풀이의 방향성을 올바르게 설정하고 검토할 수 있었다. 아울러 수열 입력에 따라 6가지 초기 상태 카드가 실시간으로 연두색 완료 상태로 변하는 동적 대시보드 구조 덕분에, 동전 상태의 대칭성과 전이 과정을 직관적으로 이해하는 데 큰 도움이 되었다.

[도움이 되지 않은 부분 (한계점)]

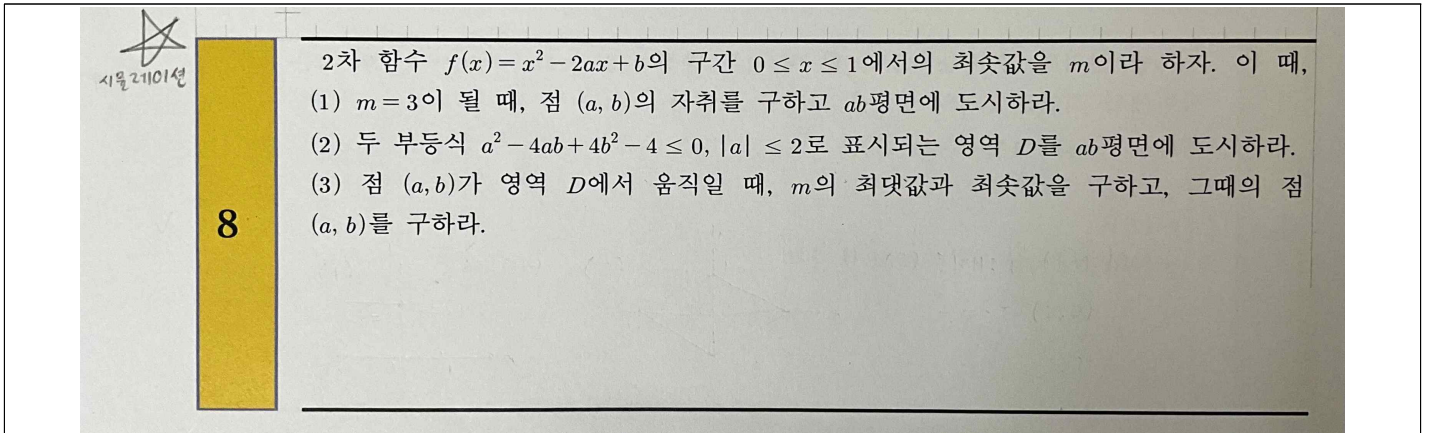
본 시뮬레이터는 입력된 수열과 조건에 따른 계산 결과, 즉 도달 여부와 최소 시행 횟수라는 최종적인 데이터만을 도출해 줄 뿐이다. 왜 특정 수열을 제시했을 때 확률이 1이 되는지나 왜 n 의 최솟값이 3이나 7로 결정될 수밖에 없는지에 대한 수학적 인과관계나 논리적 증명 구조 자체를 텍스트나 수식으로 설명해 주지는 못한다. 즉, 알고리즘적 전수조사를 통한 정답의 확인은 가능했으나, 그것이 내포하는 수학적 원리와 본질적인 개념을 스스로 정립하는 영역에 있어서는 시뮬레이터 동작만으로 해결할 수 없다는 태생적인 한계가 존재했다.

[한계점에 대한 대안]

시뮬레이터가 제공한 확실한 데이터 힌트를 정답으로 삼아 역추적해 보았다. 시뮬레이터가 찾아낸 최적의 수열 조합을 토대로 동전의 상태 변화를 다이어그램으로 시각화하여 구조화하였다. 이를 통해 각 단계마다 독립적인 초기 상태들이 어떻게 겹치지 않고 효율적으로 목표 상태로 수렴해 가는지를 대수적으로 증명해 냈다. 이처럼 시뮬레이터의 강력한 연산 결과와 인간의 논리적 추론을 상호보완적으로 결합함으로써 프로그램이 가진 한계점을 극복해낼 수 있었다고 생각한다.

[두 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 두 번째 문제 (캡처해서 붙여넣기 해도 됨)



2차 함수 $f(x) = x^2 - 2ax + b$ 의 구간 $0 \leq x \leq 1$ 에서의 최솟값을 m 이라 하자. 이 때,

- (1) $m = 3$ 이 될 때, 점 (a, b) 의 자취를 구하고 ab 평면에 도식하라.
- (2) 두 부등식 $a^2 - 4ab + 4b^2 - 4 \leq 0$, $|a| \leq 2$ 로 표시되는 영역 D 를 ab 평면에 도식하라.
- (3) 점 (a, b) 가 영역 D 에서 움직일 때, m 의 최댓값과 최솟값을 구하고, 그때의 점 (a, b) 를 구하라.

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

본 문제는 대칭축의 위치에 따라 제한된 구간 내에서 이차함수의 최솟값이 가변되는 특성을 지니고 있다. 매개변수 a 와 b 의 변화에 따라 최솟값 m 의 대수적 형태가 세 가지 구간으로 분기되는데, 이러한 수리적 변화를 단순히 식만으로 추적하는 것은 직관적인 이해에 한계가 존재한다. 대칭축 $x=a$ 가 구간의 왼쪽에 있을 때, 내부에 있을 때, 그리고 오른쪽에 있을 때에 따라 최솟값을 결정하는 기하학적 요인이 함수의 형태로 달라지기 때문에, 이를 실시간으로 시각화하여 관찰할 수 있는 도구가 필요했다. 특히 문항 (2)와 (3)처럼 매개변수 평면 ab 상에서 복잡한 연립부등식 영역 D 를 정의하고, 그 영역 안에서 움직이는 점 (a, b) 의 위치에 따른 m 의 극값을 구하는 과정은 고도의 공간적인 추상성을 요구한다. 영역 D 의 경계선과 최솟값 m 의 자취가 만나는 접점이나 교점의 위치를 인간이 손으로 작도하여 최대·최소를 판별하는 것은 계산 실수의 위험이 존재한다. 따라서 AI 플랫폼을 통해 제작한 시뮬레이터로 a 와 b 의 값을 슬라이더로 미세하게 제어하면서 함수 그래프의 움직임과 매개변수 영역 내의 자취 변화를 동기화하여 모니터링할 필요가 있었다. 이러한 동적 시뮬레이션을 통해 복잡한 영역 내에서의 최적화 메커니즘을 시각적으로 명확하게 증명하고, 스스로 도출한 수리적 결론을 완벽하게 상호 검증할 수 있기 때문에 시뮬레이션이 필요하다고 느꼈다.

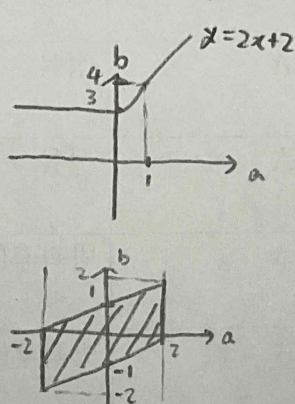
8. $f(x) = (x-a)^2 + b - a^2$

(1) i) $a < 0 \rightarrow m = b$
 ii) $0 \leq a \leq 1 \rightarrow m = f(a) = b - a^2$
 iii) $a > 1 \rightarrow m = f(1) = 1 - 2a + b$

(2) $(a-2b)^2 - 4 \leq 0, -2 \leq a-2b \leq 2$
 $\frac{1}{2}a - 1 \leq b \leq \frac{1}{2}a + 1$
 $-2 \leq a \leq 2$

(3) i) 최댓값
 $b = a^2 + k, b' = 2a = \frac{1}{2}$
 $\therefore a = \frac{1}{4} \rightarrow b = \frac{9}{8}$
 $M = -(\frac{1}{4})^2 + \frac{9}{8} = \frac{17}{16}, (\frac{1}{4}, \frac{9}{8})$

ii) 최솟값
 $b = \frac{1}{2}a - 1, a = 2 \rightarrow (2, 0)$
 $m = -2 \times 2 + 1 = -3$



4. 사용한 AI 플랫폼을 모두 적으세요.

Gemini

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

(첫 번째 문제 해결 시의 대화를 이어나감)

이 문제를 해결하는 데에 필요한 정보를 얻게 해주는 시뮬레이션을 html로 작성하고 싶은데, 계획을 로드맵으로 세워줘. 그 다음 작성한 로드맵의 내용을 바탕으로 시뮬레이터를 작성해줘. 단, 결과물은 반드시 html 파일 1개로 나와야만 해.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

가장 눈에 띄는 점은 동적 데이터 출력 레이아웃의 고정성과 가독성 부족이었다. 최초 시뮬레이터의 구간 내 최솟값 m 을 출력하는 컨테이너 영역은 내부 텍스트의 길이가 늘어날 때를 대비한 예외 처리가 결여되어 있다. 연산 결과가 단순히 실수 형태로만 출력할 때에는 문제가 없으나, 특정 조건에서 정밀한 분수 기호와 주석이 결합된 복잡한 문자열이 주입될 경우, 텍스트가 컨테이너의 가로 폭을 초과하여 UI가 깨지거나 다음 줄로 강제 개행되는 현상이 발생한다. 해당 데이터 디스플레이 블록 내부의 정렬 방식을 유연하게 다듬고, 텍스트의 줄바꿈을 강제로 방지

하는 스타일 스펙 및 내부 레이아웃의 자동 크기 조절 구조를 보완해야 한다.

둘째, **이차함수 구간 최적화** 시뮬레이션 엔진은 컴포넌트 간의 충돌을 방지하는 유연한 공간 스페이스 관리의 미흡이다. 가로 분할 배치를 위해 수평 정렬 플렉스 박스가 적용된 매개변수 제어 하단 위젯 레이아웃에서, 데이터 표시 영역과 조건 판별 배치가 가로 공간을 동등하게 나누어 갖도록 설계되어 있다. 이 과정에서 브라우저 창의 크기가 축소되거나 전체 화면 해상도가 변경될 때 조건 판별 배치 내부의 텍스트가 고정된 가로 폭에 갇혀 원치 않게 잘리거나 레이아웃 배치가 밀리는 문제가 발생한다. 특정 해상도 환경에서도 데이터 컴포넌트가 고유의 텍스트 공간을 온전히 확보할 수 있도록 플렉스 컨테이너 내부의 넘침 제어와 배치 요소의 자체 줄바꿈 방지 처리를 선제적으로 구축해 주어야 한다.

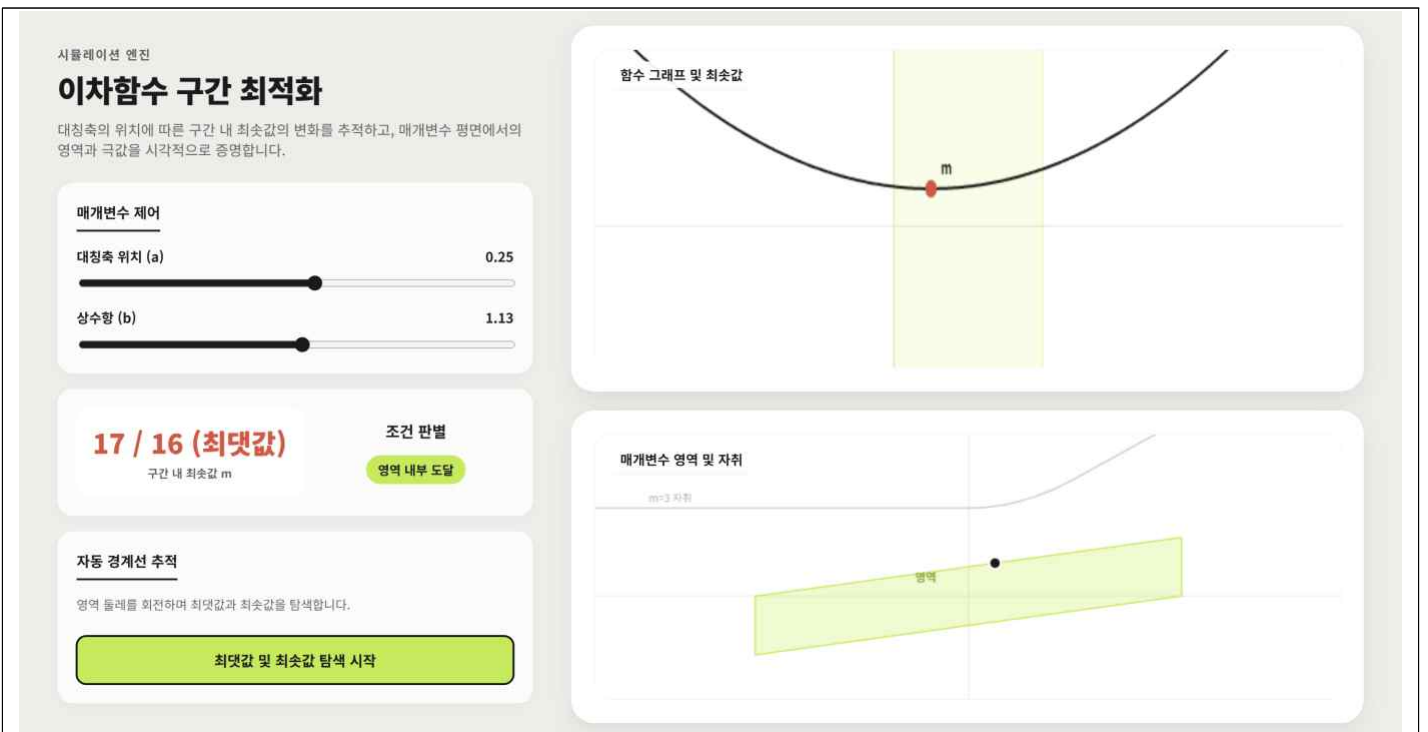


8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

(2번째)

최댓값 도출 시 텍스트가 한 줄을 넘어가는 현상을 수정해줄래?

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

1. 매개변수 제어를 통한 함수 변형

화면 좌측 중앙의 매개변수 제어 섹션에 위치한 두 개의 슬라이더를 마우스로 드래그하여 이차함수 $f(x) = x^2 - 2ax + b$ 의 계수를 실시간으로 가변한다.

- 대칭축 위치 (a) 슬라이더: 함수의 대칭축 위치를 -3부터 3까지 조정한다. 이를 움직이면 우측 상단의 함수 그래프 캔버스에서 포물선이 좌우로 동적 이동한다.
- 상수항 (b) 슬라이더: 함수의 y절편과 직결되는 상수항을 -3부터 5까지 조정한다. 이를 움직이면 포물선이 상하로 이동한다.

2. 실시간 데이터 및 영역 조건 판별 모니터링

매개변수를 변형함에 따라 좌측 하단 대시보드 박스에서 수리적 핵심 연산 데이터가 실시간으로 동기화되어 출력된다.

- 구간 내 최솟값 m: 정의된 제한된 구간 $0 \leq x \leq 1$ 내에서 함수가 갖는 실제 최솟값을 계산하여 수치로 보여준다. 만약 연산 결과가 최댓값이나 최솟값에 도달하면 텍스트 색상이 각각 빨간색과 파란색으로 반전되며 별도의 주석이 표시된다. 만약 출력된 문자열이 길어지더라도 UI 깨짐 없이 한눈에 확인할 수 있다.
- 조건 판별: 현재 설정된 매개변수 쌍 (a, b)가 문제에서 제시한 연립부등식 영역 D 내부에 존재하는지 실시간으로 판별한다. 영역 내에 있을 때는 연두색으로 활성화되며 영역 내부 도달로 표시되고, 영역을 벗어나면 영역 이탈로 변경된다.

3. 우측 기하학적 캔버스 분석

우측에 가로 폭 최적화 형태로 배치된 두 개의 독립된 캔버스를 통해 대수적 수식을 기하학적으로 증명한다.

- 함수 그래프 및 최솟값: xy평면 상에 이차함수 그래프가 작도된다. 문제의 기준이 되는 구간 $0 \leq x \leq 1$ 영역은 연두색 음영으로 하이라이트 처리되어 있으며, 대칭축 위치 a의 분기에 따라 해당 구간 내에서 결정되는 실제 최솟값 m의 위치가 빨간색 점과 문자 레이블로 실시간 추적된다.
- 매개변수 영역 및 자취: ab평면 상에 매개변수의 변동 범위가 시각화된다. 문제 (2)번에서 요구하는 평행사변형 형태의 영역 D가 연두색 다각형으로 정확히 표현되어 있으며, 문항 (1)번의 조건인 $m=3$ 인 점들의 자취가 연한 검은색 곡선과 직선의 조합으로 덧그려져 있어 현재 나의 매개변수 좌표(검은색 점)와 영역 간의 기하학적 위치 관계를 직관적으로 비교 분석할 수 있다.

4. 자동 경계선 추적 기능 활용

매개변수 평면의 경계선을 따라 움직이며 극값을 자동으로 탐색하는 알고리즘을 실행하여 효율적인 전수조사를 수행 가능하다.

- 좌측 최하단의 최댓값 및 최솟값 탐색 시작 버튼을 클릭한다.
- 시스템이 영역 D의 둘레를 자동으로 회전하며 점을 이동시키고, 이론적인 최댓값인 점 $(1/4, 9/8)$ 과 최솟값 지점인 $(2, 0)$ 에 도달하면 연산을 잠시 멈추고 데이터를 화면에 강조하여 보여준다.
- 자동 탐색을 도중에 중단하고 싶다면 빨간색으로 활성화된 '탐색 중단' 버튼을 누르면 즉시 정지하며, 슬라이더를 다시 직접 조작할 수 있는 수동 제어 모드로 복귀한다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

[도움이 된 부분]

대칭축의 위치 분기에 따른 최솟값 m의 대수적 변화와 ab평면 상의 기하학적 영역을 직관적으로 연계하여 파악하는 데 결정적인 도움이 되었다. 문항 (1)에서 $m=3$ 일 때의 자취가 직선과 포물선이 결합된 형태로 나타나는 이유를 슬라이더 조작을 통해 직접 확인하며 풀이의 방향성을 정립할 수 있었다. 특히 문항 (2)와 (3)의 연립부등식 영역 D가 ab평면 상에서 평행사변형으로 작도되는 모습을 시각적으로 확인하고, 자동 경계선 추적 기능을 실행하여 영역 D의 테두리를 따라 최댓값과 최솟값을 탐색함으로써 이론적으로 도출했던 최댓값과 최솟값 및 그때의 좌표쌍 (a, b)가 완벽한 정당임을 확신하고 검토할 수 있었다.

[도움이 되지 않은 부분 (한계점)]

시뮬레이터는 수치 계산과 기하학적 작도를 실시간으로 투영해 줄 뿐, 각 문항을 해결하기 위한 대수적 분기 조건식의 유도 과정이나 판별식 처리에 대한 수리 논리적 단계를 스스로 전개해 주지는 못한다. 예를 들어 대칭축 a 의 범위가 0 미만, 0 이상 1 이하, 1 초과일 때 왜 최솟값 m 의 함수식이 각각 다르게 도출되는지에 대한 근본적인 배경 원리는 스크립트 연산 결과만으로는 설명되지 않는다. 즉, 정답에 대한 시각적 확신과 검증의 도구로는 훌륭했으나, 수학적 논증의 전개 과정 자체를 대체할 수는 없다는 한계가 있었다.

[한계점에 대한 대안]

시뮬레이터가 보여주는 동적 그래프의 기하학적 특징을 힌트로 삼아 역으로 수학적 엄밀성을 갖추는 대수적 증명을 수행했다. xy 평면 캔버스에서 대칭축의 위치가 제한된 구간 $[0, 1]$ 의 왼쪽, 내부, 오른쪽에 걸칠 때 포물선의 꼭짓점과 양 끝점 중 어느 곳에서 최솟값이 발생하는지 기하학적으로 관찰한 뒤, 이를 바탕으로 구간을 세 가지 케이스로 분류하여 수식을 유도했다. 또한 ab 평면의 영역 D 경계선과 m 의 자취가 만나는 접점의 메커니즘을 미분과 연립부등식의 영역 개념을 도입하여 수리적으로 완전하게 논증함으로써 시뮬레이션의 결과물과 개인의 수학적 사고를 융합했다.

[마무리]

1. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

생성형 AI는 추상적인 수학적 아이디어를 정밀한 디지털 환경으로 전환하고 인간의 물리적 한계를 보완하는 다각적인 가치를 제공한다. AI는 개발 지식이 깊지 않더라도 사용자가 요구하는 수학적 조건을 완벽히 반영한 HTML 기반 시뮬레이터를 초고속으로 설계하고 구현해 준다. 또한 첫 번째 문제의 문항 (4)처럼 수작업으로 계산하기 불가능에 가까운 방대한 경우의 수에 대한 전수조사 자동화를 컴퓨터의 연산 능력을 통해 청크 단위로 분할하여 단 몇 초 만에 처리하고 최적 해를 도출해 낸다. 나아가 매개변수의 연속적인 변화에 따라 실시간으로 변화하는 그래프와 ab 평면의 부등식 영역을 웹 캔버스 기술을 통해 정밀하게 작도하여 추상적인 대수 식을 직관적인 공간 정보로 투영해 주며, 텍스트 오버플로우 방지 처리나 비동기 멈춤 방지 루프 등 프로그램의 완성도를 높이는 실무적인 UI/UX 최적화 피드백을 실시간으로 제시해 준다.

반면 시뮬레이터가 도출한 데이터와 시각적 결과물은 수리적 현상의 관찰일 뿐, 이를 학술적인 지식과 증명으로 승화시키는 것은 오직 인간이 주도해야 하는 수학적 사고의 영역이다. 먼저 문제의 핵심 조건인 동전의 제한 상태나 이차함수의 대칭축 위치에 따른 구간 분기 등을 정확하게 파악하고, 이를 시뮬레이터가 어떤 방식으로 판별해야 하는지 논리적 메커니즘을 정의하여 AI에게 지시하는 수학적 모델링의 설계 및 프롬프트 기획을 인간이 주도해야 한다. 또한 시뮬레이터가 찾아낸 정답과 현상을 보고 왜 그러한 결론이 도출될 수밖에 없는지 상태 전이 행렬이나 구간별 부등식 영역의 경계선 분석을 통해 인과관계를 규명하고 대수적으로 증명해 내야 한다. 나아가 AI가 작성한 프로그램 코드가 문제의 엄밀한 조건을 정확히 반영했는지 논리적으로 검토하는 데이터 신뢰성 검증을 수행하고, 시뮬레이션을 통해 확인한 수리적 구조를 내재화하여 보고서의 심화 결론을 도출하고 다른 학문이나 진로 분야의 실무적 최적화 문제로 치환하여 생각하는 고차원적 통찰과 응용을 주도해야 한다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험은 유전공학 및 생물정보학 분야에서 마주하게 될 실무적 과제를 해결하는 데 매우 강력한 방법론적 시사점을 준다. 현대의 생물정보학은 유전체 데이터의 폭발적인 증가로 인해 단순한 생물학적 실험을 넘어, 방대한 데이터를 정밀한 알고리즘과 수학적 모델로 처리하는 능력이 핵심 경쟁력이 되었다. 첫 번째 시뮬레이터 제작을 통해 경험한 많은 경우의 수에 대한 전수조사와 자동화 엔진 구축 과정은 유전공학에서 수억 개의 염기서열 데이터를 정렬하고 변이를 탐색하는 서열 정렬 알고리즘의 최적화 메커니즘과 일맥상통한다. 물리적 계산의 한계를 극복하기 위해 연산 작업을 분할 처리하고 비동기 루프를 설계했던 경험은, 향후 차세대 염기서열 분석 데이터를 처리하는 대용량 생물정보학 파이프라인의 연산 효율성을 극복하고 알고리즘 병목 현상을 해결하는 실무적 기반이 된다.

또한 두 번째 이차함수 최적화 시뮬레이터를 통해 매개변수의 연속적인 변동을 추적하고 ab평면 상의 부등식 영역과 극값을 시각화해 본 경험은 생물학적 시스템의 예측 모델링에 직접적으로 투영된다. 유전공학에서 생체 네트워크를 컴퓨터로 모의실험하는 시스템 생물학이나 단백질 구조 예측, 특정 유전자의 발현 패턴 최적화 등은 수많은 미지수와 제한 조건이 결합된 고차원 방정식의 연속이다. 슬라이더를 통해 변수를 미세 제어하고 자동 경계선 추적 알고리즘으로 시스템의 임계점과 최적의 해를 찾아냈던 경험은, 유전자 교정 기술 설계 시 오프타겟 효과를 최소화하는 최적의 가이드 RNA 서열을 수학적으로 모델링하고 시뮬레이션 환경에서 먼저 검증하는 실무 능력으로 확장된다.

결과적으로 AI와의 협업을 통해 추상적인 수리 모델을 구체적인 시뮬레이션 엔진으로 구현해 낸 전 과정은 인간의 가설 설정 능력과 AI의 초고속 연산 제어 능력을 결합하는 융합적 문제해결 방식을 체득하게 해 주었다. 생물정보학 실무 연구에서 생성형 AI의 가이드라인을 활용해 복잡한 유전 데이터 분석용 스크립트를 신속하게 작성 및 실험하고, 도출된 시각적 시뮬레이션 결과를 바탕으로 생물학적 현상의 본질과 인과관계를 인간의 논리적 사고로 증명해 내는 컴퓨터 기반 연구 주도 능력을 배양하는 시사점을 제공한다.

3. 보고서를 작성하며 느낀점을 간단히 작성하세요.

두 가지 복잡한 수학적 문제를 분석하고, 각각의 확률 모델과 최적화 메커니즘을 검증하는 시뮬레이터를 직접 기획하고 구현해 보면서 다각적인 통찰을 얻을 수 있었다. 처음에는 수식을 나열하고 손으로 계산하는 전통적인 수학 풀이 방식에만 익숙해져 있었으나, 시행 횟수가 늘어남에 따라 기하학적으로 증가하는 경우의 수나 복잡한 매개변수 평면의 영역 조건을 다룰 때는 직관의 한계와 물리적인 계산 실수의 위험성에 직면하게 되었다. 이러한 한계점을 생성형 AI와의 협업을 통해 알고리즘과 실시간 시각화를 갖춘 시뮬레이터로 구체화하면서, 추상적인 대수 수학기론을 동적이고 직관적인 공간 정보로 전환하는 컴퓨터 기반 수학적 모델링의 강력한 유용성을 체감했다.

특히 이번 프로젝트는 기술에 대한 맹목적인 의존에서 벗어나, 기술과 인간의 역할 분담을 명확히 인식하는 계기가 되었다. AI가 제공하는 연산 능력과 시각화 피드백은 복잡한 문제의 힌트와 정답을 확인하는 데 탁월한 가치를 발휘했지만, 그 결과물 뒤에 숨겨진 수리적 인과관계를 규명하고 논리적인 증명 구조를 완성하는 것은 결국 인간의 주도적인 수학적 통찰력과 비판적 사고력의 영역임을 깨달았다. 이번 수행평가를 통해 AI라는 강력한 도구를 내 논리적 가설 검증에 유기적으로 결합시키는 미래 지향적인 연구 패러다임을 미리 경험해 볼 수 있었던 매우 뜻깊은 기회였다.